UNIVERSIDAD TECNOLÓGICA NACIONAL · FACULTAD REGIONAL AVELLANEDA

Tecnicatura Universitaria en Programación

Programación IV · 4º cuatrimestre · Segundo parcial

TP #2 — Red social «Confesionario»

DOCUMENTO 2 DE 3

Modelo de datos MongoDB

Colecciones usuarios, publicaciones y comentarios: esquemas de Mongoose, índices, relaciones, ejemplos y consultas/aggregations de los Sprints 1 a 4.

Materia	Programación IV (4º cuatrimestre)
Trabajo	TP #2 — Red social, temática «Confesionario» · Segundo parcial
Alcance	Sprints 1 a 4 (Semana 1 a Semana 4) · se excluyen Sprints 5 y 6
Stack	Angular · NestJS · MongoDB Atlas · JWT · Cloudinary
Integrantes	[COMPLETAR]
Fecha	26/10/2026 · Versión 1.0

CONJUNTO DOCUMENTAL (mismos nombres de entidades, campos, rutas y módulos)

1. Análisis funcional
2. **Modelo de datos MongoDB**
3. Roadmap por sprints (1 a 4)

Los textos entre [COMPLETAR] deben ser reemplazados por el equipo antes de la entrega.

Índice

1. Introducción y convenciones . . . 3

2. Diagrama de colecciones y relaciones . . . 3

3. Embebido vs. referencia . . . 4

4. Colección usuarios . . . 5

4.1 Esquema . . . 5
4.2 Índices . . . 5
4.3 Esquema de Mongoose (NestJS) . . . 6
4.4 Ejemplo de documento . . . 6

5. Colección publicaciones . . . 8

5.1 Esquema . . . 8
5.2 Índices . . . 8
5.3 Esquema de Mongoose (NestJS) . . . 9
5.4 Ejemplo de documento . . . 9

6. Colección comentarios . . . 10

6.1 Esquema . . . 10
6.2 Índices . . . 10
6.3 Esquema de Mongoose (NestJS) . . . 10
6.4 Ejemplo de documento . . . 11

7. Consultas y aggregations principales . . . 12

7.1 Listado paginado con orden (E-06) . . . 12
7.2 Filtro por usuario . . . 12
7.3 Últimas 3 publicaciones con sus comentarios (Mi Perfil) . . . 12
7.4 Dar y quitar «Me gusta» (E-09 / E-10) . . . 13
7.5 Detalle y baja lógica de publicación (E-07 / E-08) . . . 13
7.6 Comentarios paginados, alta y edición (E-11 / E-12 / E-13) . . . 14
7.7 Estadísticas por rango de fechas (S4) . . . 14
7.8 Usuarios: listado, alta y bajas lógicas (E-14 a E-17) . . . 15

8. Evolución del modelo por sprint . . . 16

8.1 Configuración de MongoDB Atlas . . . 16

9. Anexo — Nomenclatura común . . . 16

1. Introducción y convenciones

Este documento define el modelo de datos de «Confesionario» en MongoDB Atlas para los Sprints 1 a 4. Usa exactamente los nombres de colecciones, campos, rutas y módulos del Documento 1 (Análisis funcional) y del Documento 3 (Roadmap).

- Tres colecciones: **usuarios**, **publicaciones** (las confesiones) y **comentarios**. Modelos de Mongoose: Usuario, Publicacion, Comentario (decorador @Schema de @nestjs/mongoose).
- Identificadores en camelCase y sin tildes ni ñ (contrasena, descripcion, publicacion).
- **timestamps: true** en todas las colecciones: Mongoose agrega createdAt y updatedAt. createdAt es la «fecha» de publicaciones y comentarios y la base de las estadísticas (S-23).
- Bajas lógicas con booleanos: usuarios.**habilitado** y publicaciones.**activa**. Nada se borra físicamente; las consultas filtran por esos campos.
- Imágenes en Cloudinary: se guarda la URL segura (secure_url) y el public_id (para poder borrar la imagen si falla el alta).
- Fechas guardadas en UTC; las estadísticas agrupan en hora de Argentina (S-18).

> **Supuestos S-01 y S-03 ·** Temática «Confesionario» (S-01): cada documento de *publicaciones* es una confesión. Se mantiene una sola imagen opcional por publicación (S-03); si se aceptaran varias, imagenUrl / imagenPublicId pasarían a un array *imagenes: [{ url, publicId }]*.

2. Diagrama de colecciones y relaciones

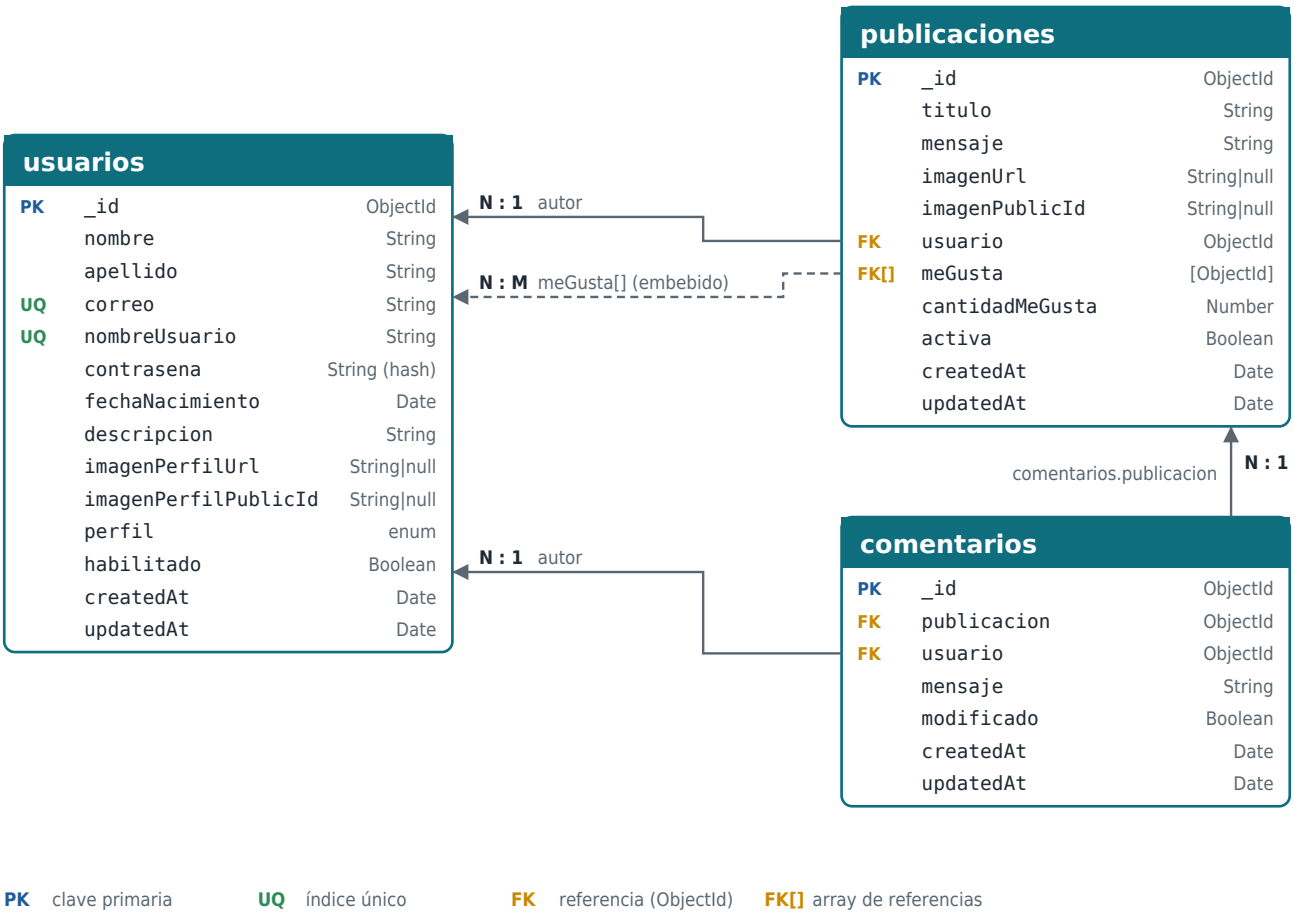


Figura 1. Colecciones, campos y relaciones. Las flechas apuntan al lado «1» de la relación.

Relación	Cardinalidad	Implementación
publicaciones.usuario → usuarios._id	N : 1	Referencia (ObjectId, ref: 'Usuario').
publicaciones.meGusta[] → usuarios._id	N : M	Array de ObjectId embebido en la publicación + contador cantidadMeGusta.
comentarios.publicacion → publicaciones._id	N : 1	Referencia (ObjectId, ref: 'Publicacion'), indexada.
comentarios.usuario → usuarios._id	N : 1	Referencia (ObjectId, ref: 'Usuario').

3. Embebido vs. referencia

Dato	Decisión	Justificación
Comentarios	**Colección propia** con referencia a publicación y usuario	Se paginan (offset/limit, más recientes primero), se cuentan por rango de fechas y por publicación (estadísticas S4) y se editan individualmente (modificado). Embebidos, la publicación crecería sin límite (tope de 16 MB por documento), la paginación requeriría $slice sobre arrays y las estadísticas obligarían a $unwind de todas las publicaciones.
Me gusta	**Embebido**: array meGusta de ObjectId + contador cantidadMeGusta	Solo se necesita saber si un usuario ya dio me gusta (un elemento del array) y cuántos hay. $addToSet evita duplicados y $pull lo quita, ambos atómicos sobre un único documento. El contador permite ordenar con un índice ({ cantidadMeGusta: -1 }) en lugar de calcular $size en cada consulta. El volumen por publicación es acotado en el TP.
Autor de la publicación / comentario	**Referencia** a usuarios	El nombre de usuario y la foto pueden cambiar; con referencia se muestran siempre actualizados mediante populate o $lookup. Además se necesita filtrar por usuario (usuarioId) y agrupar por usuario (estadísticas).
Imagen	**Campos simples** (URL + public_id)	El binario vive en Cloudinary; MongoDB guarda solo texto.

4. Colección usuarios

Guarda las cuentas de la red social: datos personales del registro, credenciales (hash), imagen de perfil, perfil (rol) y estado de habilitación.

4.1 Esquema

Campo	Tipo	Req.	Default	Validación / notas	Sprint
_id	ObjectId	sí	auto	Generado por MongoDB. Es el «id» de la payload JWT.	S1
nombre	String	sí	—	trim, 2–50 caracteres, solo letras y espacios.	S1
apellido	String	sí	—	trim, 2–50 caracteres, solo letras y espacios.	S1
correo	String	sí	—	trim, lowercase, formato email, **único** (S-22).	S1
nombreUsuario	String	sí	—	trim, 3–20, [a-zA-Z0-9._], **único** sin distinguir mayúsculas (índice con collation).	S1
contrasena	String	sí	—	Hash bcrypt (10 rondas). select: false: nunca se devuelve. La regla RN-01 se valida en el DTO antes de hashear.	S1
fechaNacimiento	Date	sí	—	Anterior a hoy (S-24).	S1
descripcion	String	no	''	trim, máx. 200 (S-06).	S1
imagenPerfilUrl	String	no	null	secure_url de Cloudinary (S-05).	S1
imagenPerfilPublicId	String	no	null	public_id de Cloudinary. No se expone en la API.	S1
perfil	String (enum)	sí	'usuario'	enum ['usuario', 'administrador']. Solo un admin asigna 'administrador' (E-15).	S1
habilitado	Boolean	sí	true	Baja/alta lógica (E-16 / E-17). Exigido en S4; conviene crearlo en S1.	S4 (adelantar a S1)
createdAt / updatedAt	Date	auto	auto	timestamps de Mongoose.	S1

4.2 Índices

Índice	Tipo	Para qué
{ correo: 1 } unique	Único	Unicidad de correo (409) y búsqueda en el login.
{ nombreUsuario: 1 } unique, collation { locale: 'es', strength: 2 }	Único	Unicidad sin distinguir mayúsculas («Ana.G» = «ana.g»).

4.3 Esquema de Mongoose (NestJS)

```
import { Prop, Schema, SchemaFactory } from '@nestjs/mongoose';
import { HydratedDocument } from 'mongoose';

export type UsuarioDocument = HydratedDocument<Usuario>;
export const PERFILES = ['usuario', 'administrador'] as const;

@Schema({ collection: 'usuarios', timestamps: true })
export class Usuario {
  @Prop({ required: true, trim: true, minlength: 2, maxlength: 50 })
  nombre: string;

  @Prop({ required: true, trim: true, minlength: 2, maxlength: 50 })
  apellido: string;

  @Prop({ required: true, trim: true, lowercase: true, unique: true })
  correo: string;

  @Prop({ required: true, trim: true, minlength: 3, maxlength: 20, match: /^[a-zA-Z0-9._]+$/ })
  nombreUsuario: string;

  @Prop({ required: true, select: false })
  contrasena: string;

  @Prop({ required: true })
  fechaNacimiento: Date;

  @Prop({ trim: true, maxlength: 200, default: '' })
  descripcion: string;

  @Prop({ type: String, default: null })
  imagenPerfilUrl: string | null;

  @Prop({ type: String, default: null })
  imagenPerfilPublicId: string | null;

  @Prop({ type: String, enum: PERFILES, default: 'usuario' })
  perfil: 'usuario' | 'administrador';

  @Prop({ default: true })
  habilitado: boolean;
}

export const UsuarioSchema = SchemaFactory.createForClass(Usuario);
UsuarioSchema.index({ nombreUsuario: 1 },
  { unique: true, collation: { locale: 'es', strength: 2 } });
```

4.4 Ejemplo de documento

```
{
  _id: ObjectId("6718a2f0c1e4b2a9d0f1a001"),
  nombre: "Ana",
  apellido: "Gómez",
  correo: "ana@mail.com",
  nombreUsuario: "ana.g",
  contrasena: "$2b$10$Qe3v1y0w8m6Jt0eS2vH8Q.7nq0bXo5vR3Yf1uJm2c9pW6s1aLk4Ze",
  fechaNacimiento: ISODate("2001-05-14T00:00:00.000Z"),
  descripcion: "Me gusta escribir de noche",
  imagenPerfilUrl: "https://res.cloudinary.com/<cloud>/image/upload/v1/red-social/perfiles/ana.webp",
  imagenPerfilPublicId: "red-social/perfiles/ana",
  perfil: "usuario",
  habilitado: true,
  createdAt: ISODate("2026-10-20T13:02:11.000Z"),
  updatedAt: ISODate("2026-10-20T13:02:11.000Z"),
  __v: 0
}
```

Nota Para que la búsqueda por nombreUsuario use el índice case-insensitive, la consulta debe llevar la misma collation: `findOne({ nombreUsuario }).collation({ locale: 'es', strength: 2 })`. Un E11000 (clave duplicada) al guardar se traduce a 409.

Nota El login selecciona explícitamente el hash: `findOne({ correo }).select('+contrasena')` y luego `bcrypt.compare(contrasena, usuario.contrasena)`.

5. Colección publicaciones

Guarda las confesiones: título, mensaje, imagen opcional, autor, me gusta (array + contador) y estado de baja lógica.

5.1 Esquema

Campo	Tipo	Req.	Default	Validación / notas	Sprint
_id	ObjectId	sí	auto	Generado por MongoDB.	S2
titulo	String	sí	—	trim, 3–100 (RN-11).	S2
mensaje	String	sí	—	trim, 1–1000 (RN-11, S-02).	S2
imagenUrl	String	no	null	secure_url de Cloudinary (S-03).	S2
imagenPublicId	String	no	null	public_id de Cloudinary. No se expone.	S2
usuario	ObjectId	sí	—	ref: 'Usuario'. Se toma del token, nunca del body.	S2
meGusta	[ObjectId]	sí	[]	ref: 'Usuario'. Sin repetidos ($addToSet). No se expone; la API devuelve meGustaDado.	S2
cantidadMeGusta	Number	sí	0	min 0. Se actualiza con $inc junto con meGusta.	S2
activa	Boolean	sí	true	Baja lógica (E-08). false oculta la publicación y sus comentarios (S-14).	S2
createdAt / updatedAt	Date	auto	auto	createdAt = fecha de la confesión (orden por fecha, estadísticas).	S2

5.2 Índices

Índice	Tipo	Para qué
{ activa: 1, createdAt: -1 }	Compuesto	Listado por fecha (E-06 orden=fecha).
{ activa: 1, cantidadMeGusta: -1, createdAt: -1 }	Compuesto	Listado por me gusta (E-06 orden=meGusta), desempate por fecha.
{ usuario: 1, activa: 1, createdAt: -1 }	Compuesto	Filtro por usuario (E-06 usuarioId; Mi Perfil).
{ createdAt: 1 }	Simple	Rango de fechas de E-18 (publicaciones por usuario).

5.3 Esquema de Mongoose (NestJS)

```
import { Prop, Schema, SchemaFactory } from '@nestjs/mongoose';
import { HydratedDocument, Types } from 'mongoose';

export type PublicacionDocument = HydratedDocument<Publicacion>;

@Schema({ collection: 'publicaciones', timestamps: true })
export class Publicacion {
  @Prop({ required: true, trim: true, minlength: 3, maxlength: 100 })
  titulo: string;

  @Prop({ required: true, trim: true, minlength: 1, maxlength: 1000 })
  mensaje: string;

  @Prop({ type: String, default: null })
  imagenUrl: string | null;

  @Prop({ type: String, default: null })
  imagenPublicId: string | null;

  @Prop({ type: Types.ObjectId, ref: 'Usuario', required: true })
  usuario: Types.ObjectId;

  @Prop({ type: [{ type: Types.ObjectId, ref: 'Usuario' }], default: [] })
  meGusta: Types.ObjectId[];

  @Prop({ default: 0, min: 0 })
  cantidadMeGusta: number;

  @Prop({ default: true })
  activa: boolean;
}

export const PublicacionSchema = SchemaFactory.createForClass(Publicacion);
PublicacionSchema.index({ activa: 1, createdAt: -1 });
PublicacionSchema.index({ activa: 1, cantidadMeGusta: -1, createdAt: -1 });
PublicacionSchema.index({ usuario: 1, activa: 1, createdAt: -1 });
PublicacionSchema.index({ createdAt: 1 });
```

5.4 Ejemplo de documento

```
{
  _id: ObjectId("6718b0aac1e4b2a9d0f1a0f3"),
  titulo: "Nunca le dije a nadie",
  mensaje: "Hoy en el colectivo le devolví la billetera a un señor y me invitó un café...",
  imagenUrl: "https://res.cloudinary.com/<cloud>/image/upload/v1/red-social/publicaciones/x1.webp",
  imagenPublicId: "red-social/publicaciones/x1",
  usuario: ObjectId("6718a2f0c1e4b2a9d0f1a001"),
  meGusta: [ ObjectId("6718a2f0c1e4b2a9d0f1a009"), ObjectId("6718a2f0c1e4b2a9d0f1a00c") ],
  cantidadMeGusta: 2,
  activa: true,
  createdAt: ISODate("2026-10-21T22:15:40.000Z"),
  updatedAt: ISODate("2026-10-21T22:40:02.000Z"),
  __v: 0
}
```

Importante meGusta y cantidadMeGusta siempre se modifican en la misma operación updateOne (ver 7.4) para que el contador nunca quede desfasado del array.

6. Colección comentarios

Guarda los comentarios de cada confesión. Es colección propia porque se paginan, se editan y se cuentan por rango de fechas (ver capítulo 3).

6.1 Esquema

Campo	Tipo	Req.	Default	Validación / notas	Sprint
_id	ObjectId	sí	auto	Generado por MongoDB.	S2 (esquema, S-09) · S3
publicacion	ObjectId	sí	—	ref: 'Publicacion'. Viene de la ruta (:publicacionId).	S2 · S3
usuario	ObjectId	sí	—	ref: 'Usuario'. Se toma del token.	S2 · S3
mensaje	String	sí	—	trim, 1–500 (RN-11).	S2 · S3
modificado	Boolean	sí	false	Pasa a true en el PUT (E-13); el front muestra «(editado)».	S3
createdAt / updatedAt	Date	auto	auto	createdAt = fecha del comentario (orden y estadísticas).	S2 · S3

6.2 Índices

Índice	Tipo	Para qué
{ publicacion: 1, createdAt: -1 }	Compuesto	Listado paginado por publicación, más recientes primero (E-11); conteo por publicación; últimos 3 en Mi Perfil.
{ createdAt: 1 }	Simple	Rango de fechas de E-19 y E-20.

6.3 Esquema de Mongoose (NestJS)

```
import { Prop, Schema, SchemaFactory } from '@nestjs/mongoose';
import { HydratedDocument, Types } from 'mongoose';

export type ComentarioDocument = HydratedDocument<Comentario>;

@Schema({ collection: 'comentarios', timestamps: true })
export class Comentario {
  @Prop({ type: Types.ObjectId, ref: 'Publicacion', required: true })
  publicacion: Types.ObjectId;

  @Prop({ type: Types.ObjectId, ref: 'Usuario', required: true })
  usuario: Types.ObjectId;

  @Prop({ required: true, trim: true, minlength: 1, maxlength: 500 })
  mensaje: string;

  @Prop({ default: false })
  modificado: boolean;
}

export const ComentarioSchema = SchemaFactory.createForClass(Comentario);
ComentarioSchema.index({ publicacion: 1, createdAt: -1 });
ComentarioSchema.index({ createdAt: 1 });
```

6.4 Ejemplo de documento

```
{
  _id: ObjectId("6718c3d1c1e4b2a9d0f1a2b7"),
  publicacion: ObjectId("6718b0aac1e4b2a9d0f1a0f3"),
  usuario: ObjectId("6718a2f0c1e4b2a9d0f1a009"),
  mensaje: "Qué lindo gesto, me alegraste el día",
  modificado: true,
  createdAt: ISODate("2026-10-21T22:30:00.000Z"),
  updatedAt: ISODate("2026-10-21T22:41:13.000Z"),
  __v: 0
}
```

Supuesto S-14 · Los comentarios no tienen campo de baja propio: el enunciado (S1–S4) no pide eliminarlos. Se ocultan cuando su publicación tiene activa = false, filtrando por la publicación (S-14).

7. Consultas y aggregations principales

Ejemplos con Mongoose dentro de los servicios de NestJS. `yo` es el ObjectId del usuario del token (decorador @UsuarioActual()). Los parámetros llegan validados por los DTOs.

7.1 Listado paginado con orden (E-06)

Excluye bajas lógicas, ordena por fecha (por defecto) o por me gusta, pagina con offset/limit y calcula meGustaDado para el usuario del token. El total se cuenta con el mismo filtro.

PublicacionesService.listar()

```
const filtro: FilterQuery<Publicacion> = { activa: true };
if (usuarioId) filtro.usuario = new Types.ObjectId(usuarioId); // filtro por usuario

const orden = q.orden === 'meGusta'
  ? { cantidadMeGusta: -1, createdAt: -1, _id: -1 }
  : { createdAt: -1, _id: -1 }; // desempate estable con _id

const pipeline: PipelineStage[] = [
  { $match: filtro },
  { $sort: orden },
  { $skip: q.offset }, // def. 0
  { $limit: q.limit }, // def. 10, máx. 50
  { $lookup: { from: 'usuarios', localField: 'usuario', foreignField: '_id', as: 'usuario',
      pipeline: [{ $project: { nombreUsuario: 1, nombre: 1, apellido: 1, imagenPerfilUrl: 1 } }] } },
  { $unwind: '$usuario' },
  { $addFields: { meGustaDado: { $in: [yo, '$meGusta'] } } },
  { $project: { meGusta: 0, imagenPublicId: 0, activa: 0, __v: 0 } },
];
if (q.comentarios > 0) pipeline.push(...etapaComentarios(q.comentarios)); // ver 7.3

const [datos, total] = await Promise.all([
  this.publicacionModel.aggregate(pipeline),
  this.publicacionModel.countDocuments(filtro),
]);
return { datos, total, offset: q.offset, limit: q.limit };
```

Nota $lookup con localField/foreignField *y* pipeline requiere MongoDB 5.0 o superior (Atlas lo cumple). Los índices compuestos del capítulo 5 cubren $match + $sort de los dos órdenes.

7.2 Filtro por usuario

Es el mismo E-06 con `usuarioId`: el $match usa el índice { usuario: 1, activa: 1, createdAt: -1 }. Si usuarioId no es un ObjectId válido, el DTO responde 400.

```
GET /publicaciones?usuarioId=6718a2f0c1e4b2a9d0f1a001&offset=0&limit=10&orden=fecha
```

7.3 Últimas 3 publicaciones con sus comentarios (Mi Perfil)

Mi Perfil llama a `GET /publicaciones?usuarioId=<yo>&limit=3&comentarios=3`. La etapa extra trae los 3 comentarios más recientes de cada publicación con su autor (S-12).

etapa opcional de E-06 (comentarios > 0)

```
const etapaComentarios = (n: number): PipelineStage[] => [
  { $lookup: {
      from: 'comentarios',
      let: { pid: '$_id' },
      pipeline: [
        { $match: { $expr: { $eq: ['$publicacion', '$$pid'] } } },
        { $sort: { createdAt: -1 } },
        { $limit: n },
```

```
        { $lookup: { from: 'usuarios', localField: 'usuario', foreignField: '_id', as: 'usuario',
            pipeline: [{ $project: { nombreUsuario: 1, imagenPerfilUrl: 1 } }] } },
        { $unwind: '$usuario' },
        { $project: { __v: 0 } },
      ],
      as: 'comentarios',
  } },
];
```

7.4 Dar y quitar «Me gusta» (E-09 / E-10)

Operaciones atómicas sobre un solo documento: la condición del filtro garantiza un único me gusta por usuario y que el contador se mueva junto con el array, aun con dos clics simultáneos.

```
// E-09 dar me gusta
const r = await this.publicacionModel.updateOne(
  { _id: id, activa: true, meGusta: { $ne: yo } },
  { $addToSet: { meGusta: yo }, $inc: { cantidadMeGusta: 1 } },
);
if (r.modifiedCount === 0) {
  const existe = await this.publicacionModel.exists({ _id: id, activa: true });
  if (!existe) throw new NotFoundException('La publicación no existe');
  throw new ConflictException('Ya diste me gusta a esta publicación'); // 409
}

// E-10 quitar me gusta
const r2 = await this.publicacionModel.updateOne(
  { _id: id, activa: true, meGusta: yo },
  { $pull: { meGusta: yo }, $inc: { cantidadMeGusta: -1 } },
);
if (r2.modifiedCount === 0) throw new NotFoundException('No habías dado me gusta a esta publicación'); //
S-13
```

7.5 Detalle y baja lógica de publicación (E-07 / E-08)

```
// E-07 detalle: mismo pipeline que 7.1 con $match { _id: id, activa: true } y $limit 1 -> 404 si vacío

// E-08 baja lógica: solo autor o administrador
const pub = await this.publicacionModel.findOne({ _id: id, activa: true }, { usuario: 1 });
if (!pub) throw new NotFoundException('La publicación no existe');
const esAutor = pub.usuario.equals(yo);
if (!esAutor && usuario.perfil !== 'administrador')
  throw new ForbiddenException('No tenés permiso para eliminar esta publicación');
await this.publicacionModel.updateOne({ _id: id }, { $set: { activa: false } });
// los comentarios quedan ocultos porque E-11/E-12/E-13 exigen publicación activa (S-14)
```

7.6 Comentarios paginados, alta y edición (E-11 / E-12 / E-13)

```
// E-11 listar: más recientes primero
await this.asegurarPublicacionActiva(publicacionId); // 404 si no existe o activa = false
const filtro = { publicacion: new Types.ObjectId(publicacionId) };
const [datos, total] = await Promise.all([
  this.comentarioModel.find(filtro)
    .sort({ createdAt: -1, _id: -1 })
    .skip(offset).limit(limit) // def. 0 / 5
    .populate('usuario', 'nombreUsuario imagenPerfilUrl')
    .select('-__v').lean(),
  this.comentarioModel.countDocuments(filtro),
]);

// E-12 agregar
await this.asegurarPublicacionActiva(publicacionId);
const nuevo = await this.comentarioModel.create({ publicacion: publicacionId, usuario: yo, mensaje });
// 201 -> se devuelve populado con el autor

// E-13 editar: solo el autor
const com = await this.comentarioModel.findOne({ _id: comentarioId, publicacion: publicacionId });
if (!com) throw new NotFoundException('El comentario no existe');
await this.asegurarPublicacionActiva(publicacionId);
if (!com.usuario.equals(yo)) throw new ForbiddenException('Solo el autor puede editar el comentario');
com.mensaje = mensaje;
com.modificado = true;
await com.save(); // updatedAt se actualiza solo
```

Nota Con paginación por offset, si alguien comenta mientras otro usuario presiona «Cargar más», el bloque siguiente puede traer un comentario ya mostrado. El front descarta repetidos por _id (Documento 3, riesgos).

7.7 Estadísticas por rango de fechas (S4)

Los tres endpoints reciben `desde` y `hasta` (AAAA-MM-DD). El DTO RangoFechasQueryDto los convierte a un rango inclusivo en hora de Argentina (S-18) y valida desde ≤ hasta (400). Solo se cuenta contenido de publicaciones activas (S-14).

RangoFechasQueryDto -> rango

```
const desde = new Date(`${q.desde}T00:00:00.000-03:00`);
const hasta = new Date(`${q.hasta}T23:59:59.999-03:00`);
const rango = { createdAt: { $gte: desde, $lte: hasta } };
```

E-18 · Publicaciones por usuario en el lapso (barras)

```
this.publicacionModel.aggregate([
  { $match: { activa: true, ...rango } },
  { $group: { _id: '$usuario', cantidad: { $sum: 1 } } },
  { $lookup: { from: 'usuarios', localField: '_id', foreignField: '_id', as: 'u',
      pipeline: [{ $project: { nombreUsuario: 1 } }] } },
  { $unwind: '$u' },
  { $project: { _id: 0, usuarioId: '$_id', nombreUsuario: '$u.nombreUsuario', cantidad: 1 } },
  { $sort: { cantidad: -1, nombreUsuario: 1 } },
]);
// -> [ { usuarioId, nombreUsuario: "ana.g", cantidad: 7 }, ... ]
```

E-19 · Comentarios realizados en el lapso (total + líneas por día)

```
this.comentarioModel.aggregate([
  { $match: rango },
  { $lookup: { from: 'publicaciones', localField: 'publicacion', foreignField: '_id', as: 'p',
      pipeline: [{ $match: { activa: true } }, { $project: { _id: 1 } }] } },
  { $match: { 'p.0': { $exists: true } } }, // descarta publicaciones dadas de baja
  { $facet: {
      total: [{ $count: 'cantidad' }],
      porDia: [
        { $group: { _id: { $dateToString: { format: '%Y-%m-%d', date: '$createdAt',
                                            timezone: 'America/Argentina/Buenos_Aires' } },
                    cantidad: { $sum: 1 } } },
        { $sort: { _id: 1 } },
        { $project: { _id: 0, fecha: '$_id', cantidad: 1 } },
      ],
  } },
]);
// -> { total: 58, porDia: [ { fecha: "2026-10-20", cantidad: 9 }, ... ] }
// ($facet devuelve total como array: el servicio lo aplana a número; días sin datos los completa el
front)
```

E-20 · Comentarios en cada publicación en el lapso (torta / dona)

```
this.comentarioModel.aggregate([
  { $match: rango },
  { $group: { _id: '$publicacion', cantidad: { $sum: 1 } } },
  { $lookup: { from: 'publicaciones', localField: '_id', foreignField: '_id', as: 'p',
      pipeline: [{ $match: { activa: true } }, { $project: { titulo: 1 } }] } },
  { $unwind: '$p' },
  { $sort: { cantidad: -1 } },
  { $limit: 10 }, // top 10 (S-18)
  { $project: { _id: 0, publicacionId: '$_id', titulo: '$p.titulo', cantidad: 1 } },
]);
// -> [ { publicacionId, titulo: "Nunca le dije a nadie", cantidad: 14 }, ... ]
```

Supuesto · «Comentarios en cada publicación en un lapso» se interpreta como los comentarios creados dentro del lapso, agrupados por publicación. Queda como pregunta al docente (Documento 3).

7.8 Usuarios: listado, alta y bajas lógicas (E-14 a E-17)

```
// E-14 listado (sin contrasena por select: false)
this.usuarioModel.find().select('-imagenPerfilPublicId -__v').sort({ createdAt: -1 }).lean();

// E-16 deshabilitar / E-17 habilitar
if (id === yo.toString()) throw new BadRequestException('No podés deshabilitarte a vos mismo'); // S-17
const u = await this.usuarioModel.findByIdAndUpdate(id, { $set: { habilitado: false } }, { new: true });
if (!u) throw new NotFoundException('El usuario no existe');

// login y JwtAuthGuard verifican habilitado -> 403 si es false (RN-07, S-16)
```

8. Evolución del modelo por sprint

Sprint	usuarios	publicaciones	comentarios
S1 · Semana 1	Todos los campos + índices únicos. habilitado se crea ya (default true) aunque se use en S4.	Módulo creado (esquema vacío o borrador).	—
S2 · Semana 2	Sin cambios.	Todos los campos e índices: titulo, mensaje, imagenUrl, imagenPublicId, usuario, meGusta, cantidadMeGusta, activa.	Esquema creado (publicacion, usuario, mensaje) para el $lookup de Mi Perfil (S-09).
S3 · Semana 3	Sin cambios.	Sin cambios.	modificado + índice { publicacion: 1, createdAt: -1 }; empiezan a crearse documentos (E-12).
S4 · Semana 4	Se usa habilitado (E-16 / E-17) y perfil «administrador» (E-15).	activa también la cambia el admin. Índice { createdAt: 1 } para estadísticas.	Índice { createdAt: 1 } para estadísticas.

8.1 Configuración de MongoDB Atlas

- Cluster gratuito; base de datos `red-social` [COMPLETAR nombre]. Usuario de base de datos con permisos de lectura/escritura solo sobre esa base.
- Network Access: el hosting gratuito del back suele no tener IP fija, por lo que normalmente se habilita 0.0.0.0/0; compensarlo con una contraseña fuerte y la URI solo en variables de entorno.
- Variable de entorno `MONGODB_URI` leída con ConfigModule. Nunca se sube al repositorio (.env en .gitignore).
- Los índices se crean con autoIndex en desarrollo; antes de la entrega verificar en Atlas (pestaña Indexes) que existan los únicos de usuarios, porque sin ellos el 409 no se dispara.

9. Anexo — Nomenclatura común

Esta tabla es idéntica en los tres documentos.

Módulos, controllers y artefactos

Concepto	Back (NestJS)	Front (Angular)	MongoDB
Autenticación	AuthModule · AuthController (/auth) · JwtAuthGuard · AdminGuard · @UsuarioActual()	AuthService · TokenService · SesionService · authGuard · adminGuard · invitadoGuard · authInterceptor · errorInterceptor	usa usuarios
Usuario	UsuariosModule · UsuariosController (/usuarios) · UsuariosService · Usuario	UsuariosService · FormularioUsuarioComponent · modelo Usuario	usuarios
Publicación (confesión)	PublicacionesModule · PublicacionesController (/publicaciones) · PublicacionesService · Publicacion	PublicacionesService · PublicacionCardComponent · NuevaPublicacionComponent	publicaciones
Me gusta	PublicacionesController (/publicaciones/:id/me-gusta)	PublicacionesService	publicaciones.meGusta[] + cantidadMeGusta
Comentario	ComentariosController (/publicaciones/:publicacionId/comentarios) · ComentariosService · Comentario	ComentariosService · ComentariosComponent · ComentarioItemComponent	comentarios
Estadísticas	EstadisticasController (/estadisticas) · EstadisticasService	EstadisticasService · DashboardEstadisticasComponent	aggregations
Imágenes	CloudinaryModule · CloudinaryService	—	*Url + *PublicId
Modales	—	ModalService · ModalComponent	—

Campos por colección y payload del token

Colección / objeto	Campos
usuarios	_id, nombre, apellido, correo, nombreUsuario, contrasena, fechaNacimiento, descripcion, imagenPerfilUrl, imagenPerfilPublicId, perfil, habilitado, createdAt, updatedAt
publicaciones	_id, titulo, mensaje, imagenUrl, imagenPublicId, usuario, meGusta, cantidadMeGusta, activa, createdAt, updatedAt
comentarios	_id, publicacion, usuario, mensaje, modificado, createdAt, updatedAt
Payload JWT	{ id, correo, nombreUsuario, perfil } · vence a los 15 minutos (exp = iat + 900 s)
Valores de perfil	«usuario» (por defecto) · «administrador»